

שלב 1: טרנספורמצית היררכיית Person איחוד ישויות אנשים)

1. **פירוק שדה שם מלא**
תחילה פוצל השדה name בטבלת Person לשני שדות נפרדים:
first_name, last_name על מנת לאפשר אחסון מובנה של שמות.
לאחר העתקת הנתונים, השדה name הוסר.
2. **אחידות שמות מפתחות ראשיים**
שונה שם עמודת id בכל אחת מהטבלאות Person, Student, Teacher לשמות ייחודיים:
personId, studentId, teacherId.
במקביל עודכנו כל מפתחות הזרים כך שיפנו לשמות החדשים.
3. **הרחבת טבלת Teacher**
נוסף השדה specialty לטבלת Teacher לצורך קליטת תחום ההתמחות של מדריכים ממסד הנתונים הישן.
4. **הטמעת ישות Parent בהיררכיית Person**
 - טבלת parents שונתה לשם Parent ושמות העמודות הותאמו לקונבנציה האחידה.
 - נתוני ההורים הועברו לטבלת Person תוך שימוש ב־offset של 10000+ למניעת התנגשויות מזהים.
 - הוסדר הקשר ההיררכי בין Parent ל־Person באמצעות מפתח זר.
5. **ייבוא תלמידים ממסד הנתונים backup2**
 - נוצרה טבלה זמנית עבור students.
 - תלמידים הוכנסו ל־Person עם offset של 20000+.
 - הוגדר הקשר בין Student ל־Parent באמצעות parentId.
6. **ניקוי יתירות נתונים**
לאחר העברת הנתונים ל־Person, הוסרו מטבלת Parent שדות שכבר אינם נדרשים
(first_name, last_name, email, phone).
- 2: **התאמת לוקיישנים, קבוצות ולוחות זמנים**
 7. **מעבר למודל ממוקד בית-ספר**
השדה location_name שונה ל־school_name בטבלת Location, כדי לשקף שכל לוקיישן מייצג בית ספר.
 8. **הפרדת פרטי זמן מקבוצות ספורט**
 - טבלת weeklysessions שונתה ל־Group_Details.
 - מידע על ימים ושעות הופרד מ־Group_Of_Sports והועבר לטבלה ייעודית.
 - נוצר sequence לניהול מזהי זמן.(timeId)
 9. **אחידות שמות ומפתחות בקבוצות ספורט**
 - השדה id ב־Group_Of_Sports שונה ל־groupId.
 - עודכנו מפתחות זרים בהתאם.
 10. **ייבוא מדריכים(Instructors)**

- מדריכים הוכנסו ל- Person עם offset של 30000+.
- במקביל נוצרו רשומות תואמות ב- Teacher, כולל תחום התמחות.

11. ייבוא קבוצות ספורט (grps)

- קבוצות הומרו ל- Group_Of_Sports עם offset של 2000+.
- הוגדרו שיוכים למדריכים ול- Sports_Class.
- עודכנו מזהי קבוצות קיימות ב- Group_Details בהתאם ל- offset.

12. קישור קבוצות לשכבות גיל

נוסף yeargroup_id ל- Group_Of_Sports, והוגדר מפתח זר ל- Year_Groups.

13. הסרת תלות ישירה בין Sports_Class ל- Location

השדה location_id הוסר מ- Sports_Class, מאחר וניהול הלוקיישן מתבצע דרך שכבות הגיל.

שלב 3: קשרי מיקום (Takes_Place)

14. יצירת טבלת Takes_Place

נוצר קשר רבים-לרבים בין Year_Groups ל- Location, המאפשר להגדיר אילו שכבות פועלות בכל בית ספר.

15. אכלוס הקשר

הוגדר שכל שכבת גיל זמינה בכל לוקיישן. (CROSS JOIN)

שלב 4: התאמת קשרי השתתפות תלמידים

16. סטנדרטיזציה של Participate_In

- שמות עמודות עודכנו ל- studentId, groupId.
- מפתחות זרים עודכנו בהתאם.

17. הוספת תאריך הצטרפות

נוסף השדה enrollment_date לניהול היסטוריית הצטרפות תלמידים לקבוצות.

שלב 5-6: ניהול קיבולת קבוצות באמצעות טריגרים

18. עדכון טריגרים ופונקציות

- פונקציה הבודקת קיבולת קבוצה לפני רישום תלמיד.
- פונקציות המעדכנות את כמות המשתתפים ואת סטטוס הקבוצה (PENDING / ACTIVE / FULL) לאחר הוספה או הסרה.

19. ייבוא רישומי השתתפות (studentgroups)

- רישומים יובאו ממסד הנתונים הישן תוך התאמת offsets.
- נשמרו תאריכי ההצטרפות המקוריים.

שלב 7: אופטימיזציית ביצועים

20. יצירת אינדקסים

נוצרו אינדקסים על מפתחות זרים מרכזיים לשיפור ביצועי שאילתות.

Stage 1: Person Hierarchy Transformation

The integration process began with restructuring the **Person hierarchy** in order to unify all human-related entities under a single inheritance-based design.

First, the name attribute in the Person table was decomposed into first_name and last_name to ensure atomic attribute storage and improve data normalization. After successfully populating the new fields, the original name column was removed.

Next, primary key columns were renamed to entity-specific identifiers (personId, studentId, teacherId) to enforce naming consistency and avoid ambiguity across the schema. All foreign key constraints were subsequently updated to reference the new column names.

The Teacher entity was extended by adding a specialty attribute, allowing the system to store instructors' areas of expertise imported from the legacy database.

The Parent entity was then migrated into the Person hierarchy. The original parents table was renamed to Parent, and all parent records were inserted into the Person table using an ID offset of 10,000 to prevent identifier collisions. A foreign key constraint was created to represent the inheritance relationship between Parent and Person.

Student records from the backup database were imported in a similar manner. Students were first loaded into a temporary table and then inserted into Person using an ID offset of 20,000. The parent–student relationship was preserved by adding a parentId foreign key to the Student table.

Finally, redundant personal attributes were removed from the Parent table, as all shared information was now stored centrally in the Person entity.

Stage 2: Location, Groups, and Scheduling Transformation

To align the schema with a school-centric model, the column location_name in the Location table was renamed to school_name.

Scheduling data was separated from group definitions by renaming the weeklysessions table to Group_Details. Day and time attributes were migrated from Group_Of_Sports into this dedicated scheduling table, and a sequence was introduced to generate unique timeld values.

The Group_Of_Sports table was standardized by renaming its primary key column to groupId and updating all related foreign key constraints accordingly.

Instructor data from the legacy database was then imported. Instructors were inserted into the Person table using an ID offset of 30,000 and simultaneously registered as Teacher entities, including their specialty information.

Sports groups were imported from the legacy grps table into Group_Of_Sports using an ID offset of 2,000. Existing group references in Group_Details were updated to remain consistent with the new identifiers.

Each sports group was associated with a Year_Group by adding a yeargroup_id foreign key, reflecting that groups belong to age-based cohorts rather than individual students.

The direct relationship between Sports_Class and Location was removed, as location management is now handled through year groups.

Stage 3: Year Group and Location Relationships

A new associative table, Takes_Place, was created to model the many-to-many relationship between Year_Groups and Location.

This table defines which year groups operate at which school locations.

The table was populated by creating all valid combinations of year groups and locations, ensuring full coverage across the system.

Stage 4: Student Enrollment Integration

The Participate_In table was standardized by renaming its columns to studentId and groupId. All legacy foreign key constraints were dropped and recreated using the updated identifiers.

An enrollment_date attribute was added to track the date on which a student joined a sports group.

Enrollment records from the legacy database were then imported, with all identifier offsets applied and original enrollment dates preserved.

Stage 5–6: Group Capacity Management Logic

To maintain data integrity and enforce business rules, all outdated triggers were removed and replaced with updated trigger-based logic.

A validation function was implemented to prevent student enrollment in groups that had reached maximum capacity. Additional triggers were created to automatically update the number of enrolled students and dynamically adjust the group status (PENDING, ACTIVE, or FULL) after insertions or deletions.

Stage 7: Performance Optimization

Indexes were created on all major foreign key columns to improve query performance and ensure efficient access to frequently joined tables.